

Lecture 15: Linked List Data Storage

Introduction

A linked list is a dynamic data structure where the size adjusts with insertions and deletions. While insertions/removals are efficient $O(1)$ if the position is known, accessing a specific element requires traversal $O(n)$. Common methods of data structures implemented with a linked list are:

- `insertFirst(item)` - Insert *item* at the beginning of the list.
- `insertLast(item)` - Insert *item* at the end. of the list.
- `insertAfter(item)` - Insert *item* after the current node.
- `insertBefore(item)` - Insert *item* before the current node
- `removeFirst()` - Remove the first element from the list.
- `removeLast()` - Remove the last element from the list.
- `remove()` - Remove the current node.
- `remove(item)` - Remove first occurrence of *item*.
- `set(index)` - Move current node to a given index.
- `contains(item)` - Check if a *item* exists in the list.
- `first()` - Access head data.
- `last()` - Access tail data.
- `node()` - Access current node data.
- `empty()` - Check if the list is empty.
- `size()` - Get size of the list.

To optimize the method implementations, three node pointer references – named *head*, *tail*, and *crnt* points to the head, tail, and a desired node of the list, respectively – and an integer named *length* are used; however, only a single node pointer pointing to the head of the list is required.

Capacity Methods

The `empty()` and `size()` both have constant time complexity $O(1)$. Their algorithms are below

<code>empty()</code>	<code>size()</code>
1. return <i>head</i> is null.	1. return <i>length</i> .

Without *length*, the `size()` method would have a linear worst-case big-oh runtime $O(n)$ since it requires traversing the list.

Access Methods

Determining if a value is present or positioning a node pointer reference in the dataset requires a traversal; hence, `contains()` and `set()` have linear time complexity $O(n)$. Their algorithms are

<code>contains(item)</code>	<code>set(idx)</code>
1. <i>crnt</i> $\leftarrow$ <i>head</i> .	1. <i>crnt</i> $\leftarrow$ <i>head</i> .
2. while <i>crnt</i> $\neq$ null and <i>crnt</i> .data $\neq$ <i>item</i> do	2. while <i>crnt</i> $\neq$ null and <i>idx</i> $>$ 0 do
1. <i>crnt</i> $\leftarrow$ <i>crnt</i> .next.	1. <i>crnt</i> $\leftarrow$ <i>crnt</i> .next.
3. return <i>crnt</i> $\neq$ null.	2. <i>idx</i> $\leftarrow$ <i>idx</i> - 1.
	3. return <i>crnt</i> $\neq$ null.

The remaining methods `first()`, `last()`, `node` have constant time complexity $O(1)$. Their algorithms are

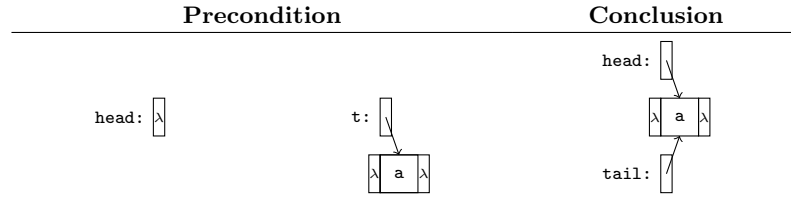
<code>first()</code>	<code>last()</code>	<code>node()</code>
1. if <i>head</i> $\neq$ null, then	1. if <i>tail</i> $\neq$ null, then	1. if <i>crnt</i> $\neq$ null, then
2.. return <i>head</i> .data	2.. return <i>tail</i> .data	2.. return <i>crnt</i> .data
2. throw out-of-bound error.	2. throw out-of-bound error.	2. throw out-of-bound error.

They throw an exception if their node pointer reference is null.

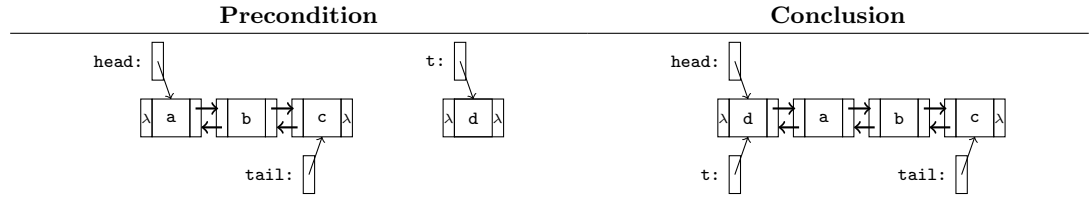
Insertion Methods

When modifying a linked list, four scenarios must be considered – modifying an empty list, the head, tail, and body of the list. At most, four link assignments are required for insertions in a doubly linked list to ensure a stable list.

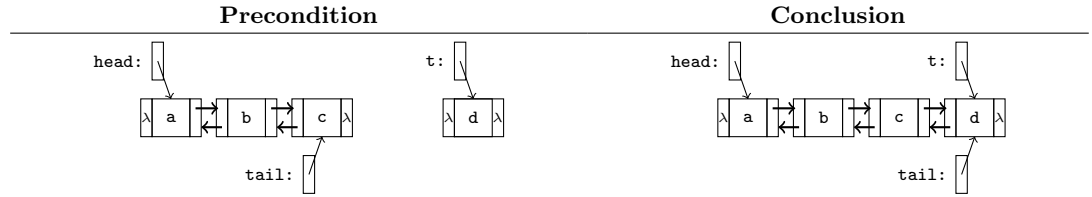
Scenario 1: An empty list. *head* is assigned *t* (the new node).



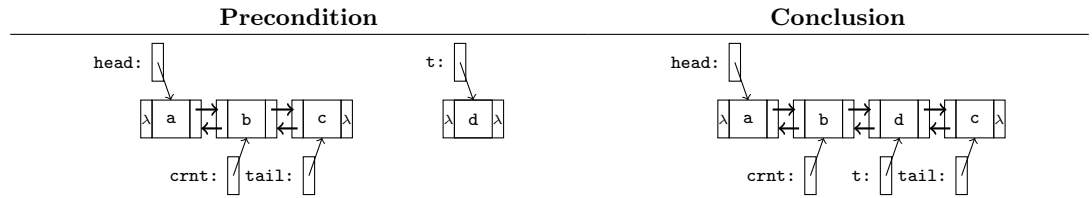
Scenario 2: The head. the next link of *t* (the new node) is assigned *head*, the previous link of *head* is assigned *t*, and then, *head* is assigned *t*.



Scenario 3: The tail. the previous link of *t* (the new node) is assigned *tail*, the next link of *tail* is assigned *t*, and then, *tail* is assigned *t*.



Scenario 4: The body. the previous link of *t* is assigned *crnt*, the next link of *t* is assigned the next link of *crnt*, the next link of the previous link of *t* is assigned *t*, the previous link of the next link of *t* is assigned *t*.



Given all scenarios, the insertion method algorithms are as follows

insertFirst(item)

1. if *head* is null, then
 1. *head* $\leftarrow$ new Node(*item*).
 2. *tail* $\leftarrow$ *head*.
2. else,
 1. *head*.previous $\leftarrow$ new Node(*item*).
 2. *head*.previous.next $\leftarrow$ *head*.
 3. *head* $\leftarrow$ *head*.previous.
3. *length* $\leftarrow$ *length* + 1.

insertLast(item)

1. if *head* is null, then
 1. *head* $\leftarrow$ new Node(*item*).
 2. *tail* $\leftarrow$ *head*.
2. else,
 1. *tail*.next $\leftarrow$ new Node(*item*).
 2. *tail*.next.previous $\leftarrow$ *tail*.
 3. *tail* $\leftarrow$ *tail*.next.
3. *length* $\leftarrow$ *length* + 1.

```
insertAfter(item)
1. if head is null, then
  1. head  $\leftarrow$  new Node(item).
  2. tail  $\leftarrow$  head.
  3. length  $\leftarrow$  length + 1.
2. elif crnt  $\neq$  null, then
  1. t  $\leftarrow$  new Node(item).
  2. t.previous  $\leftarrow$  crnt.
  3. t.next  $\leftarrow$  crnt.next.
  4. t.previous.next  $\leftarrow$  t.
  5. if t.next  $\neq$  null, then
    1. t.next.previous  $\leftarrow$  t.
  6. else, tail  $\leftarrow$  t.
  7. length  $\leftarrow$  length + 1.
```

```
insertBefore(item)
1. if head is null, then
  1. head  $\leftarrow$  new Node(item).
  2. tail  $\leftarrow$  head.
  3. length  $\leftarrow$  length + 1.
2. elif crnt  $\neq$  null, then
  1. t  $\leftarrow$  new Node(item).
  2. t.previous  $\leftarrow$  crnt.previous.
  3. t.next  $\leftarrow$  crnt.
  4. t.next.previous  $\leftarrow$  t.
  5. if t.previous  $\neq$  null, then
    1. t.previous.next  $\leftarrow$  t.
  6. else, head  $\leftarrow$  t.
  7. length  $\leftarrow$  length + 1.
```

Each method has a constant big-oh worst-case runtime $O(1)$ since the *tail* and *crnt* references are included. Additionally, the methods `insertAfter()` and `insertBefore()` perform the empty and body scenarios; however, they have an additional check for the body scenario in the case they are working with the tail and head, respectively.

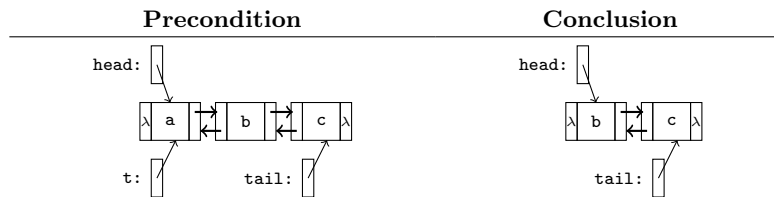
Removal Methods

At most two link assignments are needed, excluding temporary node assignment, and a possible deallocation for a removal.

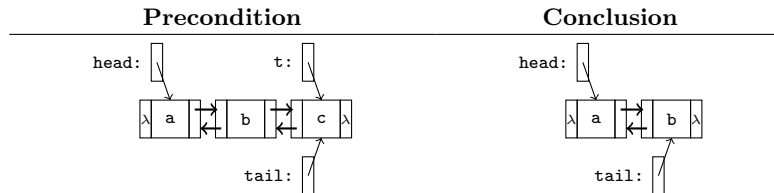
Scenario 1: An empty list. Nothing is done since there is nothing that can be removed.

Precondition	Conclusion
root: 	root: 

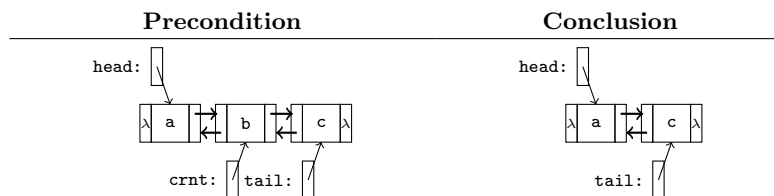
Scenario 2: The head. *t* (temporary node) is assigned *head*, *head* is assigned its next link, and then, the previous link of *head* is assigned null if *head* is not null; otherwise, *tail* is assigned null. Then, *t* is deallocated.



Scenario 3: The tail. *t* (temporary node) is assigned *tail*, *tail* is assigned its previous link, and then, the next link of *tail* is assigned null if *tail* is not null; otherwise, *head* is assigned null. Then, *t* is deallocated.



Scenario 4: The body. the next link of the previous link of *crnt* is assigned the next link of *crnt*, and the previous link of the next link of *crnt* is assigned the previous link of *crnt*. Then, *crnt* will be deallocated.



Given all scenarios, the insertion method algorithms are as follows

`removeFirst()`

1. if `head` $\neq$ null, then
 1. `t` $\leftarrow$ `head`.
 2. `head` $\leftarrow$ `head.next`.
 3. if `head` $\neq$ null, then
 1. `head.previous` $\leftarrow$ null.
 4. else, `tail` $\leftarrow$ null.
 5. deallocate `t`.
 6. `length` $\leftarrow$ `length` - 1.

`removeLast()`

1. if `tail` $\neq$ null, then
 1. `t` $\leftarrow$ `tail`.
 2. `tail` $\leftarrow$ `tail.previous`.
 3. if `tail` $\neq$ null, then
 1. `tail.next` $\leftarrow$ null.
 4. else, `head` $\leftarrow$ null.
 5. deallocate `t`.
 6. `length` $\leftarrow$ `length` - 1.

`remove()`

1. if `head` $\neq$ null and `crnt` $\neq$ null, then
 1. if `crnt.previous` $\neq$ null, then
 1. `crnt.previous.next` $\leftarrow$ `crnt.next`.
 2. else,
 1. `head` $\leftarrow$ `head.next`.
 2. if `head` $\neq$ null, then
 1. `head.previous` $\leftarrow$ null.
 3. if `crnt.next` $\neq$ null, then
 1. `crnt.next.previous` $\leftarrow$ `crnt.previous`.
 4. else,
 1. `tail` $\leftarrow$ `tail.previous`.
 2. if `tail` $\neq$ null, then
 1. `tail.next` $\leftarrow$ null.
 5. deallocate `crnt`.
 6. `length` $\leftarrow$ `length` - 1.

`remove(item)`

1. if `contains(item)`, then
 1. `remove()`.

The rightmost `remove()` method has a linear big-oh worst-case runtime $O(n)$ since it invokes `contains()`; whereas, the other methods have a constant big-oh worst-case runtime $O(1)$ since `crnt` and `tail` are included.